

Frontend Repo Structure

```
my-react-app/
├── public/
│   └── index.html
├── src/
│   ├── assets/                                # Images, icons, fonts
│   │   ├── images/
│   │   └── styles/
│   ├── components/                            # Reusable UI components
│   │   ├── Button/
│   │   │   ├── Button.jsx
│   │   │   └── Button.css
│   │   └── Navbar/
│   ├── pages/                                # Page-level components (routes)
│   │   ├── Home.jsx
│   │   ├── Login.jsx
│   │   └── Dashboard.jsx
│   ├── layouts/                              # Page layouts (optional but clean)
│   │   └── MainLayout.jsx
│   ├── hooks/                                # Custom React hooks
│   │   └── useAuth.js
│   ├── services/                             # API calls
│   │   ├── api.js
│   │   └── authService.js
│   ├── utils/                                # Helper functions
│   │   └── formatDate.js
│   ├── context/                              # React Contexts
│   │   └── AuthContext.jsx
│   ├── routes/                               # Route definitions
│   │   └── AppRoutes.jsx
│   ├── store/                                # State management (Redux/Zustand)
│   │   └── index.js
│   ├── App.jsx
│   ├── main.jsx                             # or index.js
│   └── index.css
├── .env
├── .gitignore
├── package.json
├── README.md
└── vite.config.js / webpack.config.js
```

List of required software for frontend-

- React JS
- Jest
- GitHub

GitHub Repo for frontend-

https://github.com/Reiyounaise/MF_Purchase_Optimizer_FE.git

Some useful Git commands-



Git Commands – What They Do



REPOSITORY SETUP

git init

Create a new Git repository in the current folder.

```
git init
```

git clone

Download an existing repository from GitHub.

```
git clone https://github.com/user/repo.git
```



STATUS & INFO

git status

Show current state (modified, staged, untracked files).

```
git status
```

git log

View commit history.

```
git log
```

```
git log --oneline
```

git show

Show details of a specific commit.

```
git show commit_id
```

Frontend Repo Structure

git diff

See changes between files, commits, or branches.

```
git diff
```

```
git diff --staged
```



FILE TRACKING

git add

Add files to staging area.

```
git add file.py
```

```
git add .
```

git rm

Delete file and track deletion.

```
git rm file.txt
```

git mv

Rename or move file.

```
git mv old.py new.py
```



COMMITTS

git commit

Save staged changes.

```
git commit -m "message"
```

git commit --amend

Edit last commit (message or files).

```
git commit --amend
```



BRANCHING

git branch

List, create, or delete branches.

```
git branch
```

```
git branch dev
```

```
git branch -d dev
```

git switch

Switch branches (modern).

```
git switch main
```

```
git switch -c feature-x
```

git checkout

Older version of switch (also for files).

```
git checkout dev
```

Frontend Repo Structure

```
git checkout -- file.py
```



MERGING & REBASING

git merge

Merge another branch into current branch.

```
git merge dev
```

git rebase

Move commits on top of another branch.

```
git rebase main
```

git cherry-pick

Apply a specific commit from another branch.

```
git cherry-pick commit_id
```



REMOTES (GitHub)

git remote

Manage remote repositories.

```
git remote -v
```

```
git remote add origin URL
```

git fetch

Download updates WITHOUT changing files.

```
git fetch
```

git pull

Fetch + merge from remote.

```
git pull
```

git push

Upload commits to GitHub.

```
git push
```

```
git push -u origin dev
```



STASHING

git stash

Temporarily save uncommitted changes.

```
git stash
```

git stash pop

Restore stashed changes.

```
git stash pop
```

git stash list

View stashes.

```
git stash list
```



UNDO & FIX

git reset

Move HEAD backward.

```
git reset --soft HEAD~1
```

```
git reset --mixed HEAD~1
```

```
git reset --hard HEAD~1    # dangerous
```

git revert

Undo a commit safely (new commit).

```
git revert commit_id
```

git restore

Undo file changes (modern).

```
git restore file.py
```

```
git restore --staged file.py
```



CLEANUP

git clean

Remove untracked files.

```
git clean -f
```

```
git clean -fd
```



IGNORING FILES

.gitignore

Tell Git what NOT to track.

```
node_modules/
```

```
.env
```

```
__pycache__/
```



TAGS & VERSIONS

git tag

Mark releases.

```
git tag v1.0
```

```
git push origin v1.0
```



CONFIGURATION

git config

Set username, email, editor.

Frontend Repo Structure

```
git config --global user.name "Your Name"
git config --global user.email "you@email.com"
```



ADVANCED / RARE (FYI)

Command	Purpose
git blame	Who changed each line
git bisect	Find bug-causing commit
git reflog	Recover lost commits
git archive	Create zip/tar
git submodule	Nested repos



THE 12 COMMANDS YOU *ACTUALLY* USE 90% OF THE TIME

```
git init
git clone
git status
git add .
git commit -m ""
git pull
git push
git branch
git switch
git merge
git log
git diff
```